

Cross-Variant Seed-Tree Reuse in Galas: Secret-Key Recovery and Fresh-Message Forgery from Same-Key, Same-Message S/F Signatures

Martin Feussner
Selmer Center, University of Bergen
`martin.feussner@uib.no`

1 October 2026

Provenance and review status. The attack, derivation, implementation, experiments, and initial manuscript were produced by OpenAI Codex using Daybreak Blue at Ultra reasoning effort during an AI-run audit of NGCC signature candidates, without a human first proposing this cross-variant mechanism. At the date above, no human cryptanalyst has independently verified the argument, implementation, measurements, or novelty assessment. This report is a preliminary disclosure for human review and reproduction.

Abstract

Galas supplies a short S variant and a fast F variant at each of four field sizes. At a fixed field size the variants have the same untagged public and secret key formats, the same algebraic signing relation, and deterministic signing. We show a practical specification-level cross-variant key/domain-separation defect when the same valid key signs the same adversary-chosen message once under S and once under F. The seed derivation contains no variant identifier, so both signatures use the same GGM root and IV. The F opening hides one seed in its logical tree 0. That F leaf position is an internal node of the much larger S tree and, with high probability, the S opening permits reconstruction of that node seed. The two openings therefore permit reconstruction of every F tree-0 seed. The attacker recomputes the F witness mask u_F and uses the serialized relation

$$w = d_F \oplus u_F[0..\ell), \quad k = w[0..\lambda)$$

to recover the exact secret key. No hash inversion, algebraic key search, or Fiat–Shamir forgery is needed.

At each field size, the attack recovered the exact key from one same-key, same-message pair of signatures emitted by the untouched submitted S/F interfaces. After converting the recovered normative k only to the wrapper’s documented $x \parallel k$ cache encoding, the untouched F signer produced a fresh-message signature accepted by the untouched F verifier. Offline recovery took 0.05, 0.25, 0.60, and 1.19 seconds and at most 24 MiB on an AMD EPYC host. For matched valid inputs, the original and repaired signers produced byte-identical complete serialized transcripts in all eight S/F profiles. A separately written Galas-160 implementation recovered and forged on five further keys; a 512-message sweep under the real signing distribution gave 509 complementary openings. An ideal unconditioned index model predicts success probabilities between 0.9952 and 0.9975, but these are not exact serialized-signature probabilities because grinding, opening-cap conditioning, and the shared root can correlate the challenges. Failed pairs are publicly detectable and can be retried with another common message.

The result is conditional on cross-variant key reuse and access to both signing interfaces on a common message. It does not break the ordinary isolated, single-variant EUF-CMA game,

and it does not show that deployments which enforce separate typed keys are vulnerable. The specification does not instruct users to reuse a key across S and F. It also neither prohibits that reuse nor cryptographically separates the S/F key namespaces: $\text{KeyGen}(1^\lambda)$ has no variant input and $\text{sk} = k$ has no variant tag. Binding the full instance identifier into seed derivation and the key namespace removes the attack.

1 Introduction

Galas is a VOLE-in-the-Head signature built around the Gala keyed one-way function [1, 2]. At each submitted field size $\lambda \in \{160, 256, 384, 512\}$, it offers an S variant intended to reduce signature size and an F variant intended to improve throughput. The variants change the one-tree BAVC/VOLE schedule while retaining the same Gala relation.

This flexibility interacts badly with deterministic signing. The normative key and seed derivation are parameterized by λ , the public key, the secret key, and the message, but not by the S/F label. Consequently, same-key, same-message signatures from the two variants use the same root seed in two differently sized GGM trees. A node that is a hidden leaf in the F tree is an ordinary internal node in the S tree. The S opening normally permits reconstruction of that node seed. Reinterpreting its seed under the public F leaf-expansion rule completes the F sender’s first VOLE vector, which is precisely the one-time pad applied to the signing witness.

Our contributions are as follows.

1. We prove the exact shared-node identity and the complementary-opening condition directly from the normative algorithms.
2. We give a two-query-per-attempt recovery algorithm whose success or failure can be detected using only public transcript data. On success it recovers the original λ -bit key, rather than an equivalent representation.
3. We demonstrate exact key recovery from original-interface transcripts and fresh-message forgery accepted by the untouched submitted verifier at all four field sizes. A separately written hostile reproducer confirms the Galas-160 mechanism and matched negative controls.
4. We quantify the seed-tree geometry, while separating a rigorous unconditioned index calculation from the conditioned real-signer distribution. A 512-message experiment measures the latter directly.
5. We delimit the security claim: this is a cross-variant attack under a same-key, same-message prerequisite, not a one-oracle forgery against an isolated variant.

The finding is a specification-level cross-variant key/domain-separation defect under same-key reuse. The specification does not require that reuse. We tested both a repaired specification oracle and the untouched submitted interfaces. For the ordinary valid inputs used here, all eight original S/F signers produced the same complete serialized transcripts as the oracle, and all fresh forgeries were accepted by the untouched submitted verifiers. The attack does not use the public-zero key-generation fallback or message-length truncation already reported on the NGCC site.

2 Relevant construction

2.1 Keys, witness, and deterministic signing

For a fixed even field size λ , Galas works over $\mathbb{F}_{2^\lambda}$. Algorithm 19 of the specification samples k, x , tests $\text{Good}(k, x)$, and returns

$$\text{pk} = x \parallel y, \quad \text{sk} = k, \quad y = \text{Gala}_k(x). \quad (1)$$

The algorithm is written as $\text{KeyGen}(1^\lambda)$: it has no S/F input. At a fixed λ , both variants use the same 2λ -bit public key, the same λ -bit secret key, and the same witness

$$w = (k, \sigma_0, \sigma_1, \sigma_2) \in \{0, 1\}^\ell, \quad \ell = 5\lambda/2.$$

The σ_i are deterministic inverse-norm witnesses computed from (pk, k) .

Algorithm 21 is deterministic. Given the same (pk, k, M) , it first computes

$$\mu = H_{\text{msg}}(\text{pk}, M), \quad (K_{\text{root}}, iv_{\text{pre}}) = H_{\text{seed}}(k, \mu), \quad iv = H_{\text{iv}}(iv_{\text{pre}}). \quad (2)$$

No instance name is inserted in these calls. The BAVC/VOLE layer produces a mask u , and the signature exposes

$$d = w \oplus u[0..\ell]. \quad (3)$$

The complete signature also contains the BAVC opening, final challenge Δ , iv_{pre} , and the grinding counter. Verification reconstructs the opened VOLE transcript and checks the final Fiat–Shamir equation, without learning w or u .

2.2 One-tree BAVC geometry

For a concrete variant $v \in \{S, F\}$, let τ_v be the number of logical VOLE vectors. Logical vector i has $N_{v,i} = 2^{k_{v,i}}$ leaves. All logical leaves are interleaved in one binary heap with

$$L_v = \sum_{i=0}^{\tau_v-1} N_{v,i}$$

physical leaves and $2L_v - 1$ nodes. The root is node 0 and node α has children $2\alpha + 1$ and $2\alpha + 2$. We write $\text{pos}_v(i, j)$ for the normative physical position of logical leaf (i, j) .

Starting at $K_0 = K_{\text{root}}$, every internal node expands as

$$K_{2\alpha+1} \parallel K_{2\alpha+2} = \text{TreeExpand}(K_\alpha, iv, \alpha), \quad (4)$$

where the PRG frame is

$$K_\alpha \parallel iv \parallel \text{u32le}(\alpha) \parallel \text{u32le}(\text{counter}).$$

It contains neither the variant label nor L_v . A physical leaf seed is then expanded using the variant’s public leaf tweak:

$$sd_{v,i,j} \parallel com_{v,i,j} = \text{LeafExpand}(K_{\text{pos}_v(i,j)}, iv, L_v - 1 + i). \quad (5)$$

The final challenge selects one hidden index $J_v[i]$ in each logical vector. Define the union of hidden paths

$$\mathcal{H}_v = \bigcup_{i=0}^{\tau_v-1} \text{Path}(0, \text{pos}_v(i, J_v[i])). \quad (6)$$

The BAVC opening provides sibling seeds from which every node outside $\mathcal{H}_v$ is reconstructed. In particular, it permits reconstruction of all logical-tree-0 leaf seeds except $K_{\text{pos}_v(0, J_v[0])}$.

Finally, the first logical vector is converted to the sender mask by

$$R_{0,j} = \text{VoleExpand}(sd_{v,0,j}, iv, 0), \\ R_{r+1,j} = R_{r,2j} \oplus R_{r,2j+1}, \quad u_v = R_{k_v,0,0}. \quad (7)$$

The complete VOLE commitment sets $u = u_v$ from logical vector 0.

Table 1: Normative S/F schedules and serialized signature sizes.

Instance	λ	τ	T_{open}	k_{tree}	L	bytes
Galas-160S	160	14	132	11	27,648	4,812
Galas-160F	160	20	144	8	4,096	5,964
Galas-256S	256	21	218	12	79,872	12,114
Galas-256F	256	35	232	7	4,416	15,950
Galas-384S	384	32	336	12	118,784	27,784
Galas-384F	384	48	332	8	11,520	33,384
Galas-512S	512	42	456	13	172,032	49,516
Galas-512F	512	64	445	8	15,616	59,416

3 The cross-variant identity

Lemma 1 (Shared deterministic state). *Fix λ , a valid normative key pair (pk, k) , and a message M . An S signature and an F signature generated from these values have identical μ , K_{root} , iv_{pre} , and iv .*

Proof. Every quantity in (2) is a deterministic function of the displayed arguments. The specification inserts no S/F label or full instance identifier in H_{msg} , H_{seed} , or H_{iv} . The inputs are therefore byte-for-byte equal. $\square$

Lemma 2 (Shared physical nodes). *Under the conditions of Lemma 1, a physical GGM node index α that is generated in both heaps has the same seed in the S and F executions.*

Proof. Both heaps start with the same K_0 and iv . Equation (4) is a deterministic function of (K_α, iv, α) and contains no schedule identifier. Induction down the common heap indices proves the claim. $\square$

The leaf tweak in (5) does contain L_v , but this does not separate the underlying node seeds. Once an attacker reconstructs K_α from the S opening, the attacker can apply the public F tweak to it.

Proposition 3 (Every F leaf is an internal S node). *For each of the four submitted S/F pairs, every physical F leaf position—and therefore every possible F logical-tree-0 target α^* —is the index of an internal node in the corresponding S heap.*

Proof. A binary heap with L leaves numbers its internal nodes from 0 through $L - 2$ and its leaves from $L - 1$ through $2L - 2$. Thus every F leaf index is at most $2L_F - 2$, whereas an index is internal in the S heap whenever it is strictly less than $L_S - 1$. The required strict inequality holds at every field size:

Pair	$2L_F - 2$	$L_S - 1$	$2L_F - 2 < L_S - 1$
160S/F	8,190	27,647	yes
256S/F	8,830	79,871	yes
384S/F	23,038	118,783	yes
512S/F	31,230	172,031	yes

The bound covers all F leaves, so no additional property of the interleaving map or of the selected tree-0 index is needed. $\square$

Theorem 4 (Conditional exact key recovery). *Let*

$$\alpha^* = \text{pos}_F(0, J_F[0]).$$

Suppose $\alpha^* \notin \mathcal{H}_S$. From two valid, honestly generated same-key, same-message S/F signatures and the public parameters, an attacker recovers the exact normative secret key k in linear PRG work in the two heap sizes.

Proof. The F opening reconstructs every logical-tree-0 physical seed except K_{α^*} . Proposition 3 shows that α^* is an internal node of the S heap for every submitted pair. Because $\alpha^* \notin \mathcal{H}_S$, the S opening permits reconstruction of its seed. By Lemma 2, this is the same seed required by F.

The attacker now has all $K_{\text{pos}_F(0,j)}$. Applying the public F leaf expansion in (5), followed by the public fold in (7), gives the exact u_F . The F signature contains d_F , so

$$w = d_F \oplus u_F[0..\ell), \quad k = w[0..\lambda). \quad (8)$$

The first λ witness bits are exactly the normative secret key. The algorithm uses arrays linear in $L_S + L_F$ and a constant number of public PRG expansions per reconstructed node or F tree-0 leaf. $\square$

The theorem explains the whole attack chain:

missing S/F separation $\implies$ shared GGM nodes $\implies$ complementary BAVC opening $\implies u_F \implies w \implies k \implies$ fresh-message signing.
--

No property of the Gala one-way function is weakened. Instead, combining the two openings permits reconstruction of a mask that the proof system relies on keeping hidden.

4 Success probability and retry strategy

For a fixed F target node α , define

$$c_i(\alpha) = \#\{j : \alpha \text{ is an ancestor of } \text{pos}_S(i, j)\}.$$

If the S indices $J_S[i]$ are independent and uniform before grinding and opening-cap rejection, the probability that no S hidden path contains α is exactly

$$P_\alpha = \prod_{i=0}^{\tau_S-1} \left(1 - \frac{c_i(\alpha)}{N_{S,i}}\right). \quad (9)$$

If the F tree-0 index is also independent and uniform, averaging gives

$$P_{S,F} = \frac{1}{N_{F,0}} \sum_{j=0}^{N_{F,0}-1} P_{\text{pos}_F(0,j)}. \quad (10)$$

An exact enumeration of the normative schedules gives Table 2.

Table 2: Unconditioned independent-index model. These are not claimed as exact probabilities for serialized signatures.

Pair	$N_{F,0}$	$P_{S,F}$	expected pairs $1/P_{S,F}$
160S/F	256	0.996787101321	1.00322
256S/F	128	0.995242959290	1.00478
384S/F	256	0.997515982924	1.00249
512S/F	256	0.997348606864	1.00266

The qualification matters. A real signer accepts only final challenges that satisfy the grinding predicate and whose canonical BAVC opening uses at most T_{open} sibling seeds. The S and F

challenges also arise from transcripts with a common root. Equation (10) therefore describes the unconditioned schedule geometry, not a proof of the real success probability.

The real condition is nonetheless publicly testable. After reconstructing both openings, the attacker checks whether the S opening permits reconstruction of K_{α^*} . If it does not, the pair is discarded before any secret-dependent test. The attacker asks both interfaces to sign a new common chosen message and repeats. Under the random-oracle model, the distinct input changes μ and the deterministic root except with negligible collision probability.

To measure the conditioned implementation rather than infer it from Equation (10), we generated 512 same-key S/F signature pairs on independently derived common messages. Recovery succeeded on 509 and gave a clean “unrecovered target leaf” exit on three, for an observed rate $509/512 = 0.994140625$. The sweep used 48 workers and completed in 34.10 seconds of wall time on the audit host. The experiment used one key, so it measures message-to-message behavior for that key and is not a 512-key estimate.

5 Recovery algorithm and public validation

The attack on one pair is the following.

1. Query $\text{Sign}_S(\text{pk}, k, M)$ and $\text{Sign}_F(\text{pk}, k, M)$ for the same chosen M , and verify both public signatures.
2. Parse iv_{pre} and Δ from each signature. Recompute iv , decode the opening indices, and reconstruct every node seed permitted by each canonical BAVC opening.
3. Locate $\alpha^* = \text{pos}_F(0, J_F[0])$. Use the F view for the other $N_{F,0} - 1$ physical seeds and reconstruct K_{α^*} from the S opening. If the S opening does not permit that reconstruction, request another common-message pair.
4. Apply **LeafExpand** with the F tweak $L_F - 1$ to every recovered tree-0 seed. Apply **VoleExpand** and the public XOR fold to obtain u_F .
5. Compute w and k with (8).
6. Validate the candidate without the retained secret by checking $\text{Good}(k, x) = 1$ and $\text{Gala}_k(x) = y$ for the public $\text{pk} = x \parallel y$. A valid candidate is then an ordinary signing key for either variant.

At $\lambda = 160$, the two queries return $4,812 + 5,964 = 10,776$ bytes. The two main node arrays contain

$$(2 \cdot 27,648 - 1) \cdot 20 + (2 \cdot 4,096 - 1) \cdot 20 = 1,269,720 \text{ bytes},$$

with small additional bitsets and work buffers. The algorithm is deterministic after it receives a successful pair.

Fresh-message forgery requires no special construction: run the ordinary normative signing algorithm with the recovered k on a message that was not queried. Verification acceptance is therefore both the end-to-end impact and an independent public check that the recovered string is a functioning secret key.

6 Specification-conformant and pristine-source experiments

6.1 Interface repairs and untouched-source equivalence

The official specification PDF, rather than the submitted wrapper, defines the design. The companion `reproducer/README.md` records checksums for the official specification and submission archive. The reproducer includes a specification-conformant wrapper derived from the submitted sources. Three interface differences were repaired before testing:

1. key generation requires an explicit independent seed instead of using the wrapper’s public all-zero fallback;
2. the normative secret key is k , while the source wrapper caches $x \parallel k$ because its external signing API omits pk ; and
3. the full host-size message length is passed to H_{msg} , instead of the source wrapper’s 32-bit length.

The repaired entry points implement Algorithms 19, 21, and 22 as **KeyGen**, **Sign**(pk, k, M), and **Verify**(pk, M, σ). We then extracted the official archive afresh and built all eight submitted profiles without editing their source files. For the same explicit seed and valid inputs, the normative key k equalled the suffix of the source wrapper’s cached $x \parallel k$, while the prefix equalled the public x . For every S/F profile, the repaired and original signers emitted byte-identical complete serialized signatures. Each untouched verifier accepted both transcripts, and each repaired verifier accepted the untouched-source transcript. Table 3 summarizes these checks.

Table 3: Whole-transcript comparison on matched valid inputs. “Cross-accept” means that the untouched and repaired verifiers both accepted both serialized transcripts.

Profile	bytes	SHA-256 prefix	exact equality	cross-accept
Galas-160S	4,812	9859a1242bb7d7b5	yes	yes
Galas-160F	5,964	0cacfff1200e847e	yes	yes
Galas-256S	12,114	7d73c06255b34650	yes	yes
Galas-256F	15,950	d30ee52a8b2276df	yes	yes
Galas-384S	27,784	166e2bc1bc1cdc86	yes	yes
Galas-384F	33,384	c27ae298c8b8eecd	yes	yes
Galas-512S	49,516	fda7ff0f8f5cf607	yes	yes
Galas-512F	59,416	2d5115b086ec3c26	yes	yes

The pristine experiment hashed all 552 files in the extracted reference-source tree before and after building and testing; every hash remained unchanged. Its file-I/O harness calls the submitted **sig_keygen**, **sig_sign**, and **sig_verify** functions directly. After recovery, its only conversion is to place the normative k into the wrapper’s documented $x \parallel k$ cache, using public x , before calling the untouched signer. Thus neither recovery nor verification depends materially on the repaired wrapper, and the defect is not created by any repaired deviation.

For portable reproduction, the repository reproducer also replaces the submitted SM3 rotate macro’s undefined shift by 32 when its rotation count is zero with an equivalent defined C99 operation. This cleanup leaves the fixed seed transcripts unchanged under GCC. An AddressSanitizer/UndefinedBehaviorSanitizer run of the complete 160-bit recovery, forgery, and negative-control path then completed without a diagnostic. The pristine build retained the submitted macro unchanged.

6.2 All-level results

Table 4 reports the pristine-source run at every submitted field size. Each row used one S signature and one F signature emitted by the original submitted interfaces under the same normative key on a common chosen message. The recovery program received only those two serialized signatures. It did not receive the public key, cached secret, or normative key. After recovery, the experiment compared the result to the independently retained key, signed a fresh message through the untouched F signer, and called the untouched F verifier. “Wrong message” applies that fresh signature to a different message.

Table 4: End-to-end recovery through the original submitted interfaces. Times and RSS are for offline recovery, excluding signing-oracle time.

Pair	query bytes	key bytes	time (s)	RSS (MiB)	fresh F	wrong message
160S/F	10,776	20	0.05	2	accept	reject
256S/F	28,064	32	0.25	6	accept	reject
384S/F	61,168	48	0.60	13	accept	reject
512S/F	108,932	64	1.19	24	accept	reject

Every recovered key matched both the normative k suffix of the untouched wrapper secret and the repaired oracle’s Algorithm-19 key byte for byte. The 160-bit retained and recovered key had SHA-256

01f62f9bd378ef93973c4c172ba0b804377b165244c4eb6d66319ac24e50bb76.

The 160-bit fresh-message F forgery generated through the untouched signer had SHA-256

3de6b380fa1b66f3b8e9c6f7afb68e735affdacc6bd8fc1fc087aed75fcfb952

and the untouched verifier accepted it. The untouched verifier took 0.77, 3.12, 11.06, and 33.25 seconds respectively.

6.3 Independent AI reproduction and controls

A separately instructed Codex agent wrote a new 160-bit recovery implementation against a clean extraction of the official archive. It did not reuse the initial recovery source. Across five independently seeded keys it obtained five exact key matches and five accepted fresh-message forgeries. A timed recovery took 0.06 seconds and 2,048 KiB RSS. A symbol and file-access audit found no linked key-generation, signing, OWF, retained-secret, or instrumentation input in the recovery executable. AddressSanitizer reported no error on the successful path.

The following adversarial controls had their expected outcomes:

- the F opening alone stopped at its one hidden tree-0 seed;
- pairing different keys or pairing different messages failed the shared state check and emitted no key;
- swapping the S and F signature arguments failed fixed-length parsing;
- the original chosen-message signature and the fresh forgery were each rejected when verified on a different message;
- each fresh forgery was rejected under an independently generated public key, and a signature made with an independently generated wrong secret was rejected under the target public key;
- one-bit signature mutations were rejected by Algorithm 22;
- changing the serialized d_F changed the recovered string, which then failed to produce an accepted fresh signature; and
- in that independent 160-bit experiment, signatures produced by the original source and repaired oracle were byte-identical for both variants.

Separately, the pristine-source validation established whole-transcript equality for all eight S/F profiles and the all-level untouched-verifier results in Tables 3 and 4. This separates the original separately written 160-bit implementation from the later all-profile validation.

This second implementation was also produced within the AI audit and is not independent human verification. Its results confirm the practical specification-level cross-variant key/domain-separation defect under same-key reuse; they do not establish an isolated single-variant EUF-CMA break.

6.4 Reproduction

The companion repository contains a deterministic reproducer. From the `reproducer` directory, run

```
./run.sh.
```

The command generates one Galas-160 key, obtains one S signature and one F signature on the same chosen message, recovers the exact key from the two serialized signatures, and produces a fresh-message F signature. It also requires rejection of wrong-message, wrong-key, mutated-signature, and nonmatching-transcript controls. A successful run ends with `minimal_reproducer=pass`.

The optional command `./run_all_levels.sh` repeats exact recovery and fresh-message forgery at 256, 384, and 512 bits and ends with `all_levels_reproducer=pass`. The file `reproducer/README.md` records the required tools, expected running time, source-archive checksum, and the separate untouched-source validation procedure. The recovery program receives only the two serialized signatures; comparison with the retained original key is a post-recovery experiment control.

7 Security impact and limits

The demonstrated impact is exact recovery of k , followed by an ordinary fresh-message signature accepted by the normative verifier. It is stronger than transcript malleability and is not merely a reduction in a security estimate. It also recovers the original key, not only a proof-system witness or equivalent key. The prerequisites and nonclaims are equally concrete.

Cross-variant oracle access. The adversary needs a geometrically successful pair consisting of one S and one F signature under the same public/secret pair (pk, k) and on the same message. Each attempt uses two signing queries, and the pair’s geometric success is publicly testable. The ordinary EUF-CMA game for one fixed algorithm instance offers only one signing oracle. This attack therefore does not break Galas-160S alone, Galas-160F alone, or any other isolated variant in that strict game.

Classification and same-key prerequisite. This is a specification-level cross-variant key/domain-separation defect under same-key reuse. The specification does not explicitly instruct users to reuse keys across S and F. It defines $\text{KeyGen}(1^\lambda)$ without an S/F input, gives both variants the identical encodings in (1), states that the algebraic relation is the same, and defines no key-usage prohibition. The repaired and submitted core signers accept the same serialized key material in both variants. Cross-variant reuse is therefore neither prohibited nor cryptographically separated by the specified algorithms, key namespace, or serialization. An API or deployment that generates and enforces a separate typed key for each variant avoids the prerequisite.

Same-message requirement. Different messages change μ and hence the root. Repeated signing of one message within a single variant returns the same deterministic transcript and hides the same paths, so it does not provide complementary information. The attack neither claims nor needs a same-variant nonce-reuse weakness.

Success estimates. Table 2 is a rigorous calculation only for independent uniform decoded indices before the signer’s grinding and T_{open} conditioning. The 509/512 experiment supports practical success for one 160-bit key but does not prove the all-key probability or exact independence between retries. A failed pair does not yield a key in this procedure and is handled by a publicly selected retry.

No primitive cryptanalysis. The recovery relies on exact equality of PRG inputs, public BAVC reconstruction, and the witness-mask equation. It assumes neither a collision nor a weakness in the NGCC hash/XOF, the Gala relation, QuickSilver, or the Fiat–Shamir transform. The already public zero-seed and message-length source findings are unrelated.

8 Repair

The root cause is a missing full-instance domain separator at the point where long-lived secret material derives the deterministic commitment state. A direct repair is

$$(K_{\text{root}}, iv_{\text{pre}}) = H_{\text{seed}}(\text{instanceID} \parallel k \parallel \mu), \quad (11)$$

where `instanceID` canonically includes the field size and S/F label. Binding the same identifier into H_{msg} and every tree, leaf, and VOLE expansion frame provides defense in depth and makes cross-protocol equality visibly impossible at each layer.

Keys and APIs should also be typed by the complete instance. A serialized key created for Galas-160S should be rejected by a Galas-160F signing entry point even though the algebraic relation has the same shape. Known-answer tests should assert that equal raw (pk, k, M) inputs under S and F produce different roots and IVs. Either independent roots or enforced disjoint key namespaces blocks the attack; deploying both is safer and easier to audit.

9 Prior art and novelty boundary

The public NGCC Galas page listed two critical implementation reports at the 1 October 2026 checkpoint: a public-zero key-generation fallback and 32-bit message-length truncation [3]. Neither report concerns shared S/F roots, complementary BAVC openings, or recovery of u and k .

Chosen-protocol attacks established the broader warning that a key may be secure inside either of two protocols yet fail when both protocols expose the same key material [4]. Modified-parameter attacks later made the closely related point that security under one parameter set need not survive use of the same key under another [5]. These works provide the general cross-protocol and cross-parameter perspective. They do not identify the Galas shared-node identity, the complementary S/F openings, or recovery of the VOLE witness mask; those are the Galas-specific claims here.

FAEST, the closest transcript template cited by Galas, derives its root and IV from secret and message material together with a fresh λ -bit ρ by default; deterministic operation sets $\rho = 0$ [6]. This comparison illustrates why fresh or domain-separated state prevents accidental equality, but it is not prior art for the attack proved here.

Targeted searches covered the NGCC reports and security index, the local public NGCC harness snapshot, the PKC Forum, the IACR ePrint Archive, the Galas design paper title and authors, and combinations of “Galas”, “cross-variant”, “same key”, “seed tree”, and “parameter attack”. We found no public report of complementary Galas S/F openings recovering the VOLE witness mask. This is a best-effort negative search as of 1 October 2026, not proof that unpublished or unindexed prior art does not exist. The live sources were last checked on 1 October 2026.

10 Conclusion

Two Galas signatures can expose an entire signing key when the same valid key signs the same chosen message once under S and once under F. Missing variant separation makes their deterministic GGM nodes identical; the larger S opening normally permits reconstruction of the one F tree-0 node seed hidden by the F opening. Public expansion then recovers u_F , the masked witness, and the exact k . Full-parameter experiments recovered 20-, 32-, 48-, and 64-byte keys in at most 1.19 seconds and used them to produce fresh-message signatures accepted by the untouched submitted verifiers.

The precise scope should remain attached to the result. It is a practical cross-variant family attack under same-key, same-message access, while the isolated single-variant EUF-CMA games remain unbroken by this method. Full-instance domain separation and typed keys remove the shared state that enables recovery.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

- [1] Guoxiao Liu, Zhuoqing Peng, Chongxu Ren, Guowei Liu, Juntong Lin, Yimeng Sun, Keting Jia, and Hongbo Yu, *Galas Signature Scheme: Submission to ICCS NGCC*, 26 June 2026.
- [2] Guoxiao Liu, Yimeng Sun, Zhuoqing Peng, Chongxu Ren, Guowei Liu, Juntong Lin, Keting Jia, and Meiqin Wang, *GALAS: Compact VOLE-in-the-Head Signatures from Subfield Compression*, ACM CCS 2026, accepted Cycle 1, to appear.
- [3] NGCC Security Reports, *Galas (sign-12)*, <https://ngcc.dev/reports/sign-12.html>, accessed 1 October 2026.
- [4] John Kelsey, Bruce Schneier, and David Wagner, *Protocol Interactions and the Chosen Protocol Attack*, Security Protocols, 5th International Workshop, Lecture Notes in Computer Science 1361, pp. 91–104, Springer, 1998, <https://doi.org/10.1007/BFb0028162>.
- [5] Nick Howgrave-Graham, Joseph H. Silverman, Ari Singer, and William Whyte, *Modified Parameter Attacks: Practical Attacks Against CCA2 Secure Cryptosystems, and Countermeasures*, IACR Cryptology ePrint Archive, Report 2004/344, 2004, <https://eprint.iacr.org/2004/344>.
- [6] The FAEST Team, *FAEST Specification, Version 2.0*, <https://faest.info/faest-spec-v2.0.pdf>.